# Introduction to Python

Write real Python programs from zero

### Part A · Python basics

Running code, variables and types, operators, strings

### Part B · Decisions and data

if and loops; list, tuple, dict and set

### Part C · Building programs

Functions, modules, errors, a final project

**Ibrahim Sobh, PhD** · ibrahimsobh.github.io/introduction-to-python

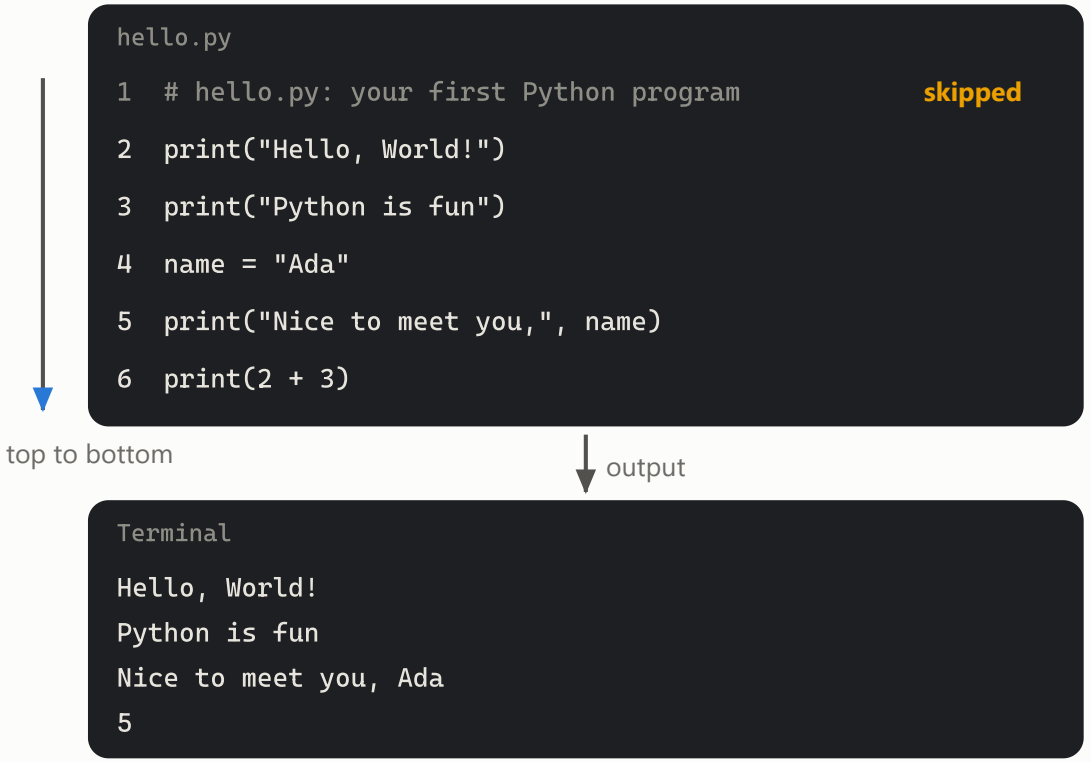
PART A

# Python basics

1 Getting Started
2 Variables and Data Types
3 Operators and Expressions
4 Strings

# A program is a list of instructions, run top to bottom

- Python reads almost like English: one of the easiest languages to learn, and one of the most widely used
- The computer follows a program one line at a time, from top to bottom
- `print()` shows values on the screen: `print(2 + 3)` shows `5`
- A line starting with `#` is a **comment**: a note for humans that Python skips




KEY INSIGHT

**Python runs your lines in order, one at a time, from top to bottom.**

# Run a whole script, or try one line at a time in the REPL

| | Script | REPL |
|---|---|---|
| **Start it with** | `python hello.py` | `python` |
| **Runs** | the whole file, top to bottom | one line at a time |
| **Shows values** | only what you `print()` | every expression's value |
| **Best for** | real programs you keep | quick experiments |

```
Script: runs the whole file
$ python hello.py
Hello, World!
Python is fun
…
```

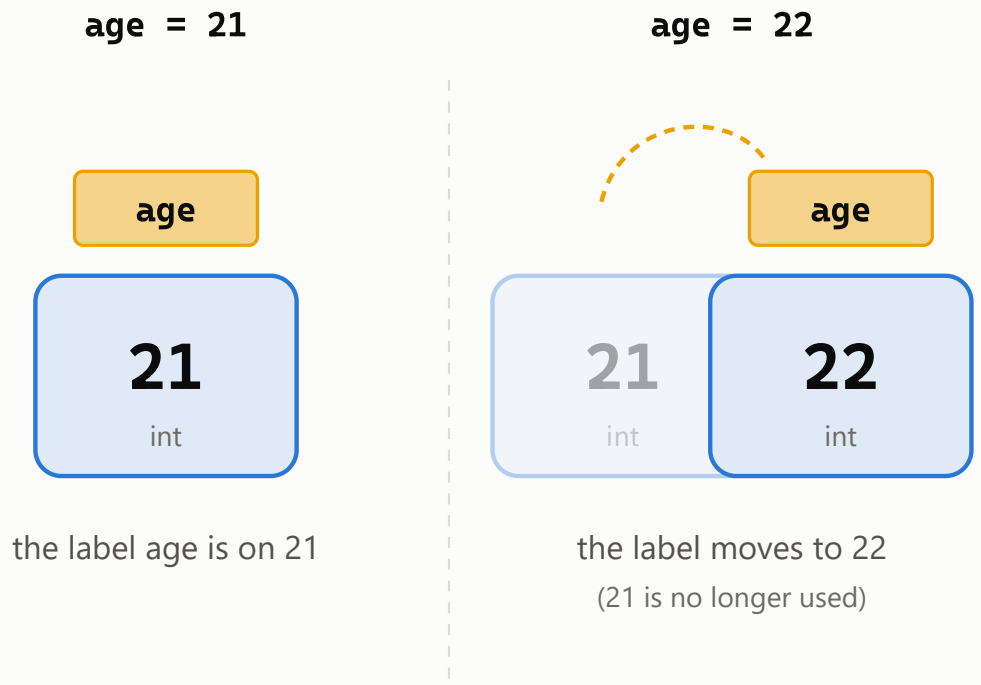
```
REPL: one line at a time
$ python
>>> 2 + 3
5
>>> name = "Ada"                shows nothing
>>> name
'Ada'
```

KEY INSIGHT

**Keep programs in .py files; use the REPL as a calculator and to test ideas.**

# A variable is a name that refers to a value

- `age = 21` stores 21 and sticks the label `age` on it
- Writing `age` later means "the value in that box"
- Give the label a new value and it simply moves
- Names: letters, digits and `_`; can't start with a digit; case-sensitive; no keywords like `if`




**KEY INSIGHT**

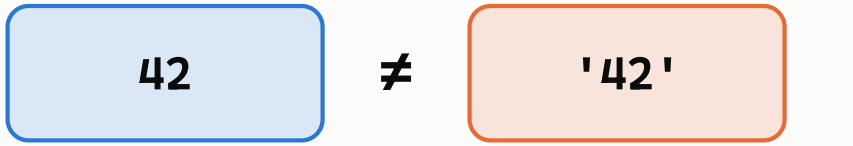
**`=` stores a value under a name; using the name reads it back.**

# Every value has a type that decides what you can do with it

| Type | What it holds | Examples |
|---|---|---|
| `int` | whole numbers | `7`, `-3`, `2025` |
| `float` | numbers with a decimal point | `3.14`, `2.0` |
| `str` | text, inside quotes | `"hello"`, `'42'` |
| `bool` | yes or no | `True`, `False` |
| `None` | "no value yet" | `None` |




**Same digits, different type**




`int: 42 + 1 is 43`

`str: '42' + 1 is a TypeError`

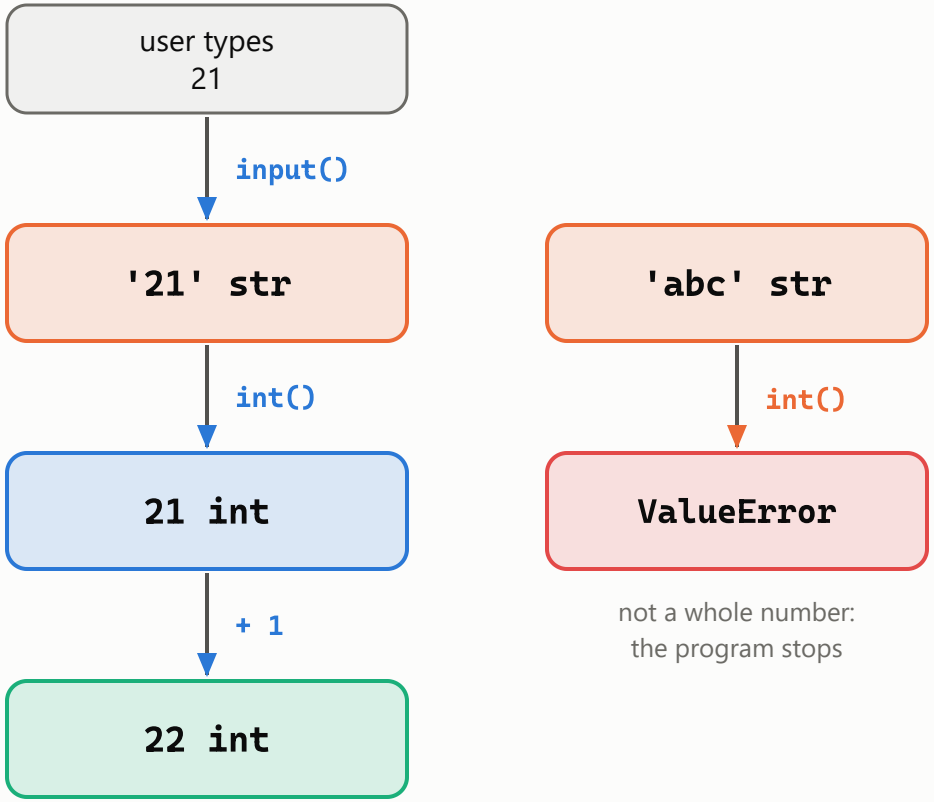
```
type(3.14)  →  <class 'float'>
```

KEY INSIGHT

**The type, not how a value looks, decides what Python can do with it: `'42'` is text.**

# input() always gives text: convert it before doing maths

- `input()` waits for the user and always returns a `str`, even if they typed digits
- Convert with `int()` or `float()`; turn a number into text with `str()`
- `"5" + 3` is a TypeError: Python won't add text to a number
- `int("5") + 3` gives `8`; `"5" + str(3)` gives `'53'`

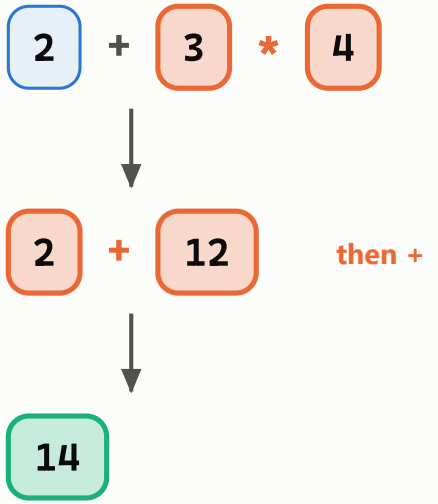



**KEY INSIGHT**

**Data from users, files and websites arrives as text. Convert it first.**

# Python works an expression out one operator at a time

- An **expression** is anything Python can work out to a single value
- `2 + 3 * 4` → `2 + 12` → `14`: `*` comes before `+`
- `/` always gives a float (`7 / 2` is `3.5`)
- `//` is whole-number divide, `%` the remainder: `7 // 2` is 3, `7 % 2` is 1




evaluated one operator at a time

**Order, first to last**

| | |
|---|---|
| 1 | `( )` brackets |
| 2 | `**` power |
| 3 | `* / // %` multiply, divide |
| 4 | `+ -` add, subtract |
| 5 | `== != < > <= >=` compare |
| 6 | `not, and, or` logic |

KEY INSIGHT

**Same rules as school maths. When in doubt, add brackets.**

# Comparisons and logic answer yes-or-no questions

- Comparisons `== != < > <= >=` give `True` or `False`: `5 > 3` is `True`
- `and`: True only if both sides are True
- `or`: True if at least one side is True
- `not`: flips True and False
- One `=` stores a value; two `==` ask a question

| a | b | a and b | a or b |
|---|---|---|---|
| True | True | True | True |
| True | False | False | True |
| False | True | False | True |
| False | False | False | False |

`x = 5`

one = stores a value

`x == 5 → True`

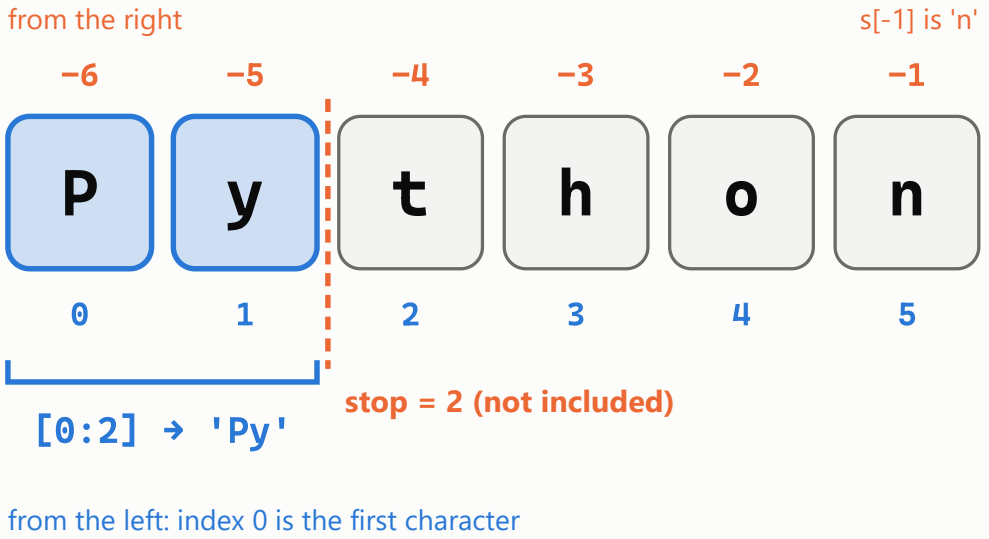
two == ask a question

**KEY INSIGHT**

**Comparisons and logic turn questions into True or False, ready for decisions.**

# A string is a row of characters, numbered from 0

- `s[0]` is the first character, `s[-1]` the last
- `s[start:stop]` goes up to but not including stop: `"Python"[0:2]` is `'Py'`
- Leave a number out for "from the start" or "to the end": `s[:3]`, `s[2:]`
- A step: `s[::2]` is `'Pto'`, `s[::-1]` is `'nohtyP'`

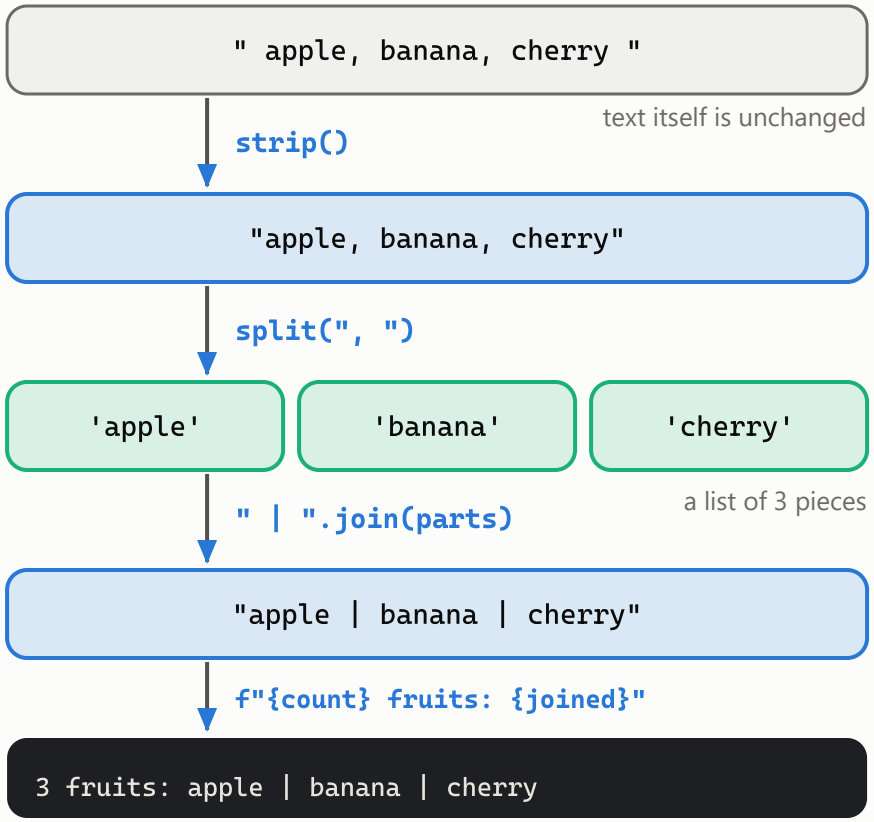



KEY INSIGHT

**Indexes start at 0, and a slice stops just before its stop index.**

# String methods give back new strings; f-strings build text

| Method | Example | Result |
|---|---|---|
| `strip()` | `" hi ".strip()` | `'hi'` |
| `split(sep)` | `"a,b,c".split(",")` | `['a', 'b', 'c']` |
| `sep.join(list)` | `"-".join(["a", "b"])` | `'a-b'` |
| `replace(old, new)` | `"cat".replace("c", "b")` | `'bat'` |
| **f-string** | `f"{name} scored {score}"` | `'Ada scored 9'` |




KEY INSIGHT

**Methods never change the original string; f-strings are the easiest way to build text.**

PART B

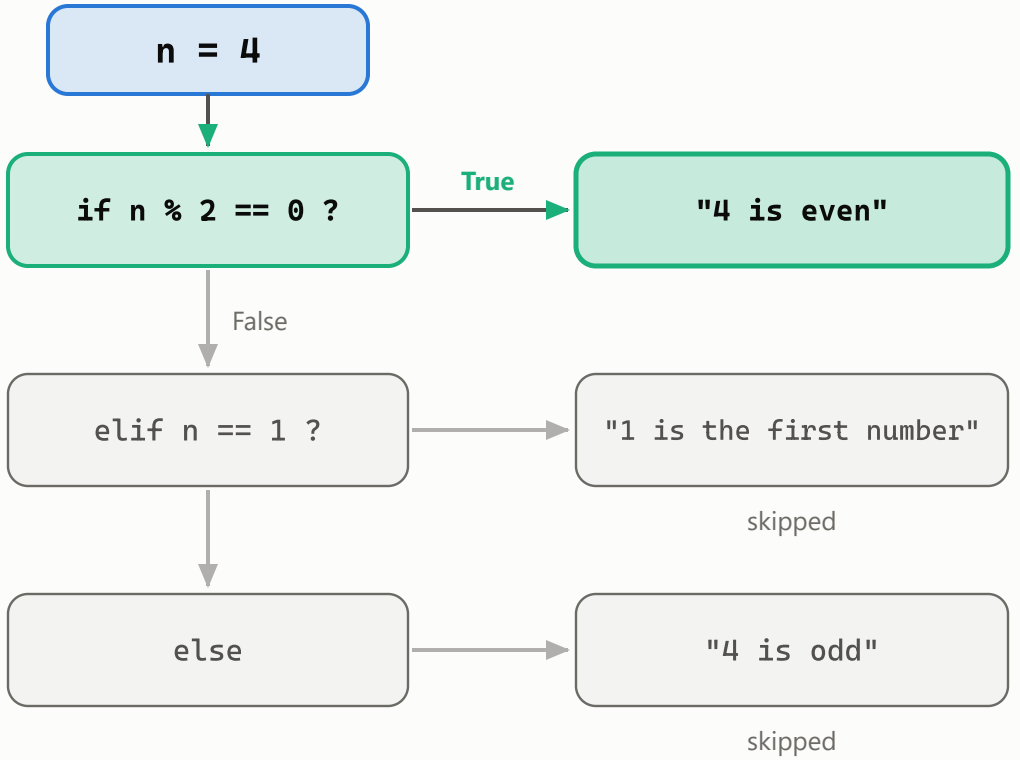
# Decisions and data

5 Control Flow

6 Data Structures

# if / elif / else: the first true condition wins

- A **condition** is an expression that is True or False, such as `age >= 18`
- Python checks the conditions from the top and runs the block under the first True one
- `else` runs only when none of them were True
- Indentation (4 spaces) is how Python knows which lines are inside a block



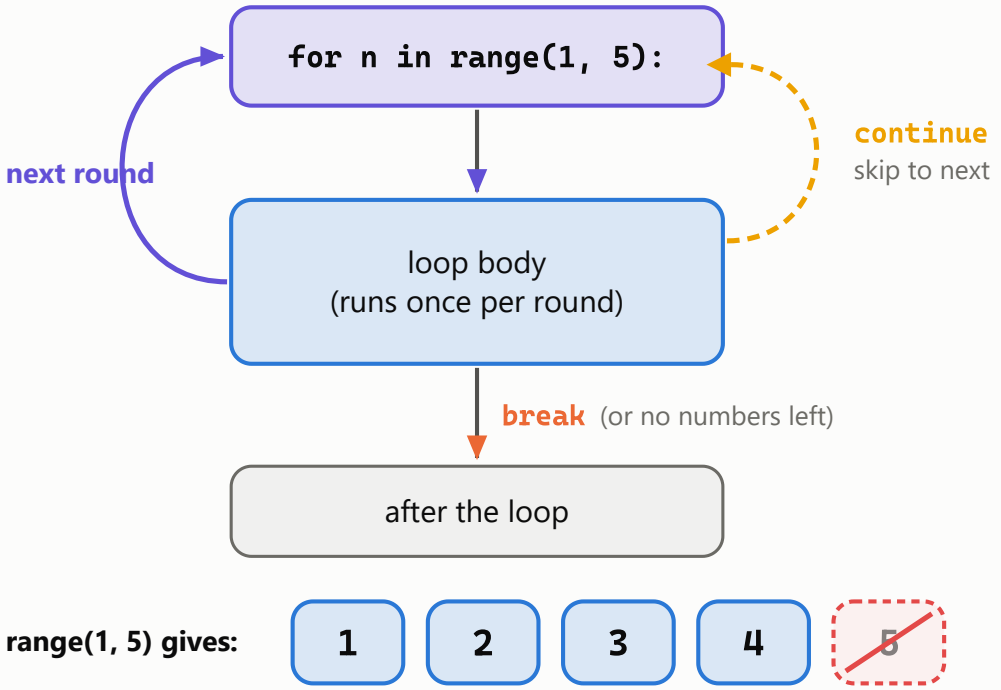

green = the path taken for n = 4; grey = never checked or skipped

KEY INSIGHT

**Only one branch runs: the first condition that is True.**

# Loops repeat a block: for over items, while until a condition is False

- `for n in range(1, 5):` repeats for 1, 2, 3, 4 (stops before 5)
- `while condition:` repeats as long as the condition stays True
- `break` leaves the loop now; `continue` skips to the next round
- A `while` loop must change something, or it runs forever



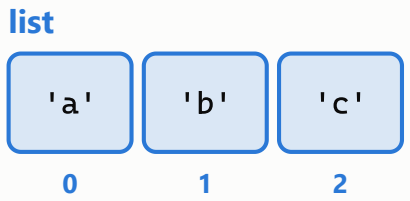

stops before 5: four rounds, n = 1, 2, 3, 4

KEY INSIGHT

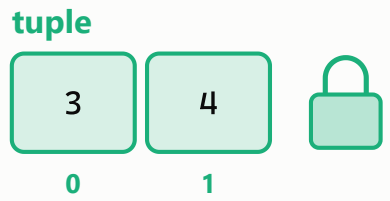
**Use for when you know what to loop over; use while to repeat until something happens.**

# Four collections, four jobs

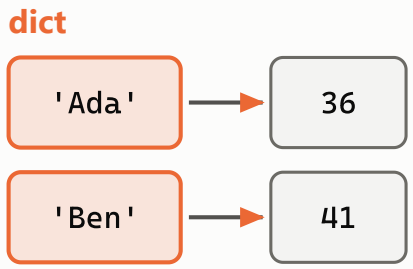
| | Ordered? | Can change? | Duplicates? |
|---|---|---|---|
| **list** `["a", "b"]` | yes | yes | yes |
| **tuple** `(3, 4)` | yes | no | yes |
| **dict** `{"Ada": 36}` | keeps insertion order | yes | keys are unique |
| **set** `{"red", "blue"}` | no | yes | no |



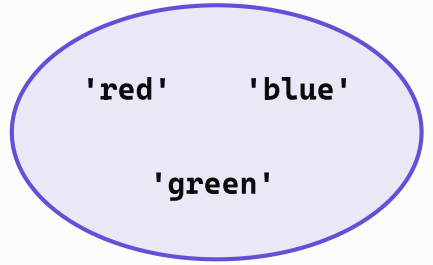

numbered slots you can change




fixed once written




key → value



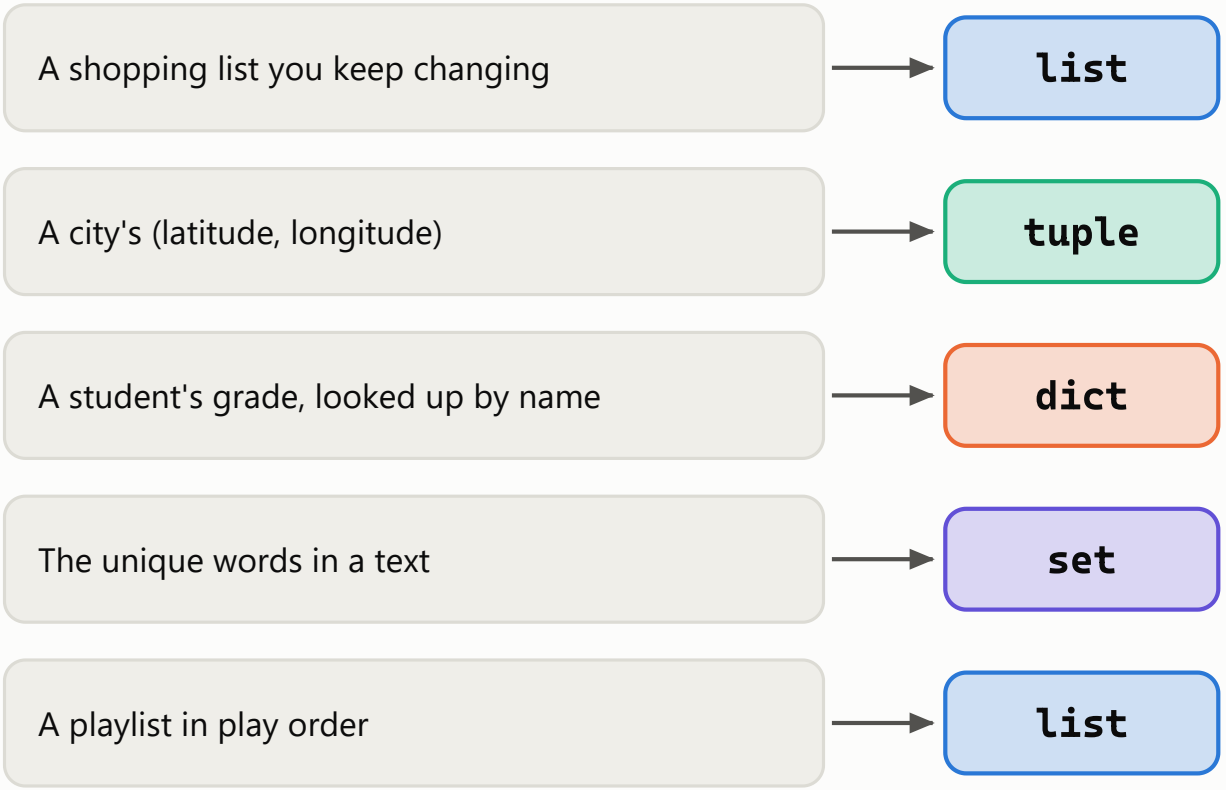

no order, no duplicates

KEY INSIGHT

**Each collection is good at a different job: pick by what you need to do.**

# Choose the collection that fits the job

- Order matters and items change → **list**
- A fixed group of values → **tuple**
- Look a value up by its key → **dict**
- Only unique items → **set**




KEY INSIGHT

**Order and change → list; fixed → tuple; look up by key → dict; unique → set.**

PART C

# Building programs

7 Functions

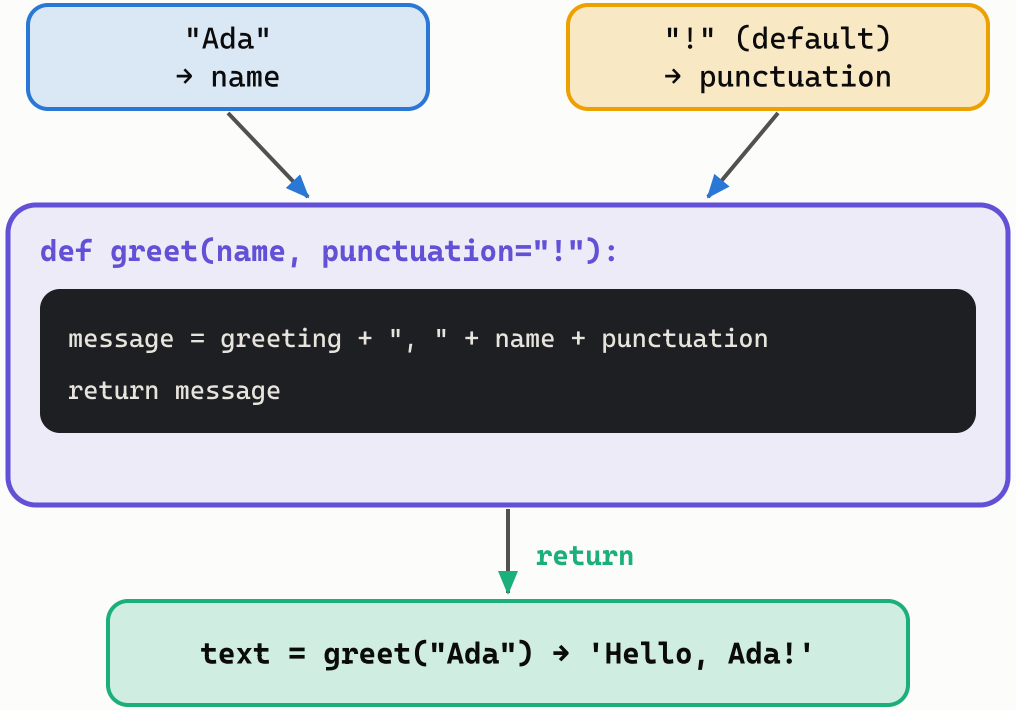
8 Modules

9 Errors and Exceptions

10 Final Project

# A function: define it once, call it as often as you like

- `def` starts a definition; the indented lines are its body
- Defining does not run it; calling does: `greet("Ada")` gives `'Hello, Ada!'`
- Arguments fill the parameters in order; defaults fill in what's left out: `area(3)` uses `height=1`
- `return` hands a value back; `print()` only shows it



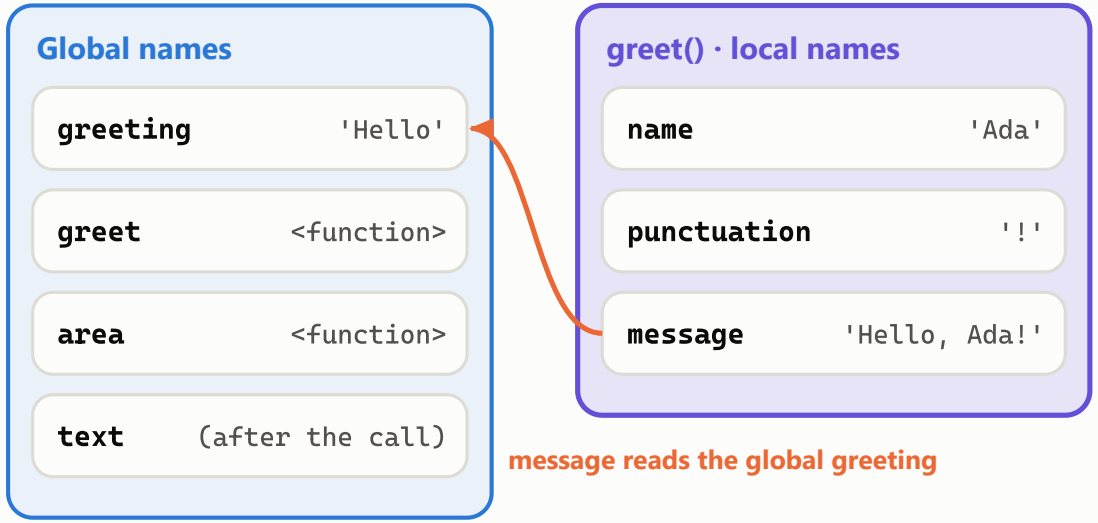

defined once, called as often as you like

KEY INSIGHT

**A function packs one job behind a name, so you write it once and reuse it.**

# Names created inside a function are local to that call

- Each call gets its own box of **local** names, which disappears when the call returns
- Names at the top level of the file are **global**
- A function can read a global name; outside code can't see a function's local names
- Prefer parameters in and `return` out over changing globals



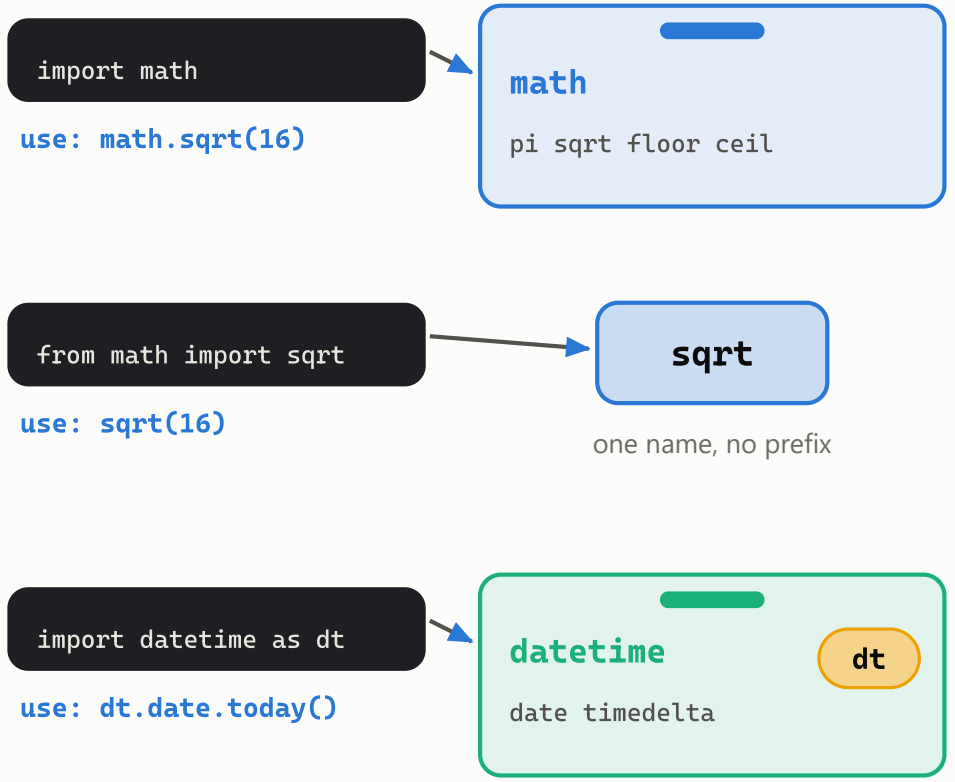

When greet() returns, the purple box disappears: name, punctuation and message are gone, and text gets the returned 'Hello, Ada!'.

KEY INSIGHT

**Local names live only while their call runs.**

# Modules: import ready-made code instead of writing it

| You write | Then you use | Good when |
|---|---|---|
| `import math` | `math.sqrt(16)` | the usual choice: it's clear where sqrt comes from |
| `from math import sqrt` | `sqrt(16)` | you use one or two names a lot |
| `import datetime as dt` | `dt.date.today()` | the module name is long |

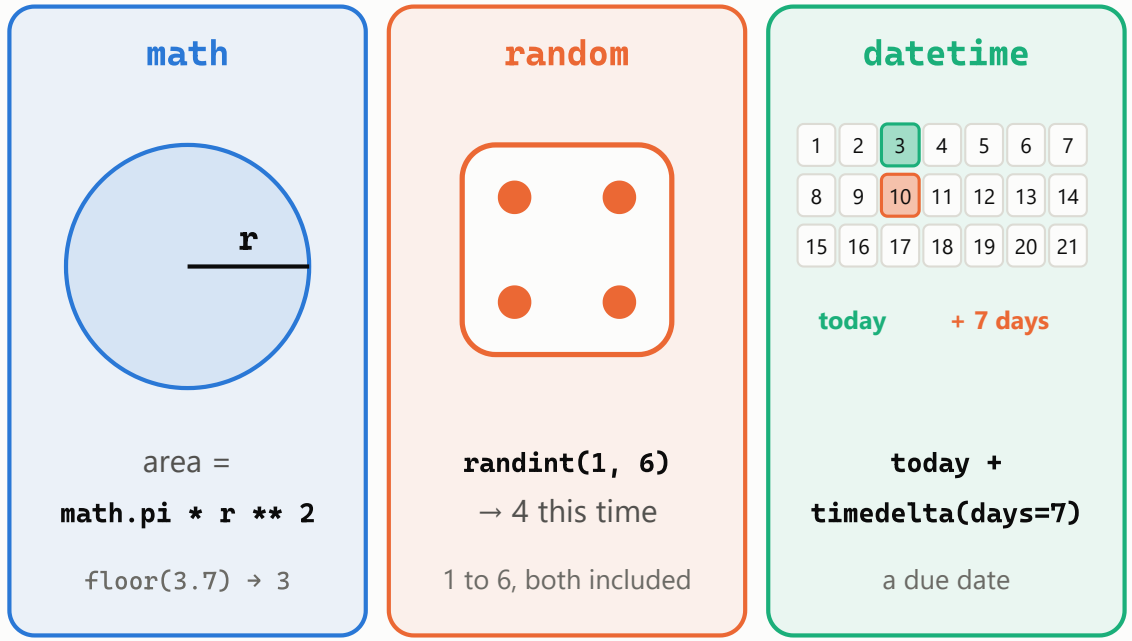



KEY INSIGHT

**A module is a .py file of ready-made code; import loads it.**

# The standard library already does the common jobs

- `math`: `math.pi`, `sqrt()`; `floor(3.7)` is 3, `ceil(3.7)` is 4
- `random`: `randint(1, 6)` like a die (both ends included), `choice()`, `shuffle()`, `sample()`
- `datetime`: `date.today()`, plus `timedelta(days=7)` for a due date
- More: files (`os`, `pathlib`), the web (`urllib`), data (`json`, `csv`)



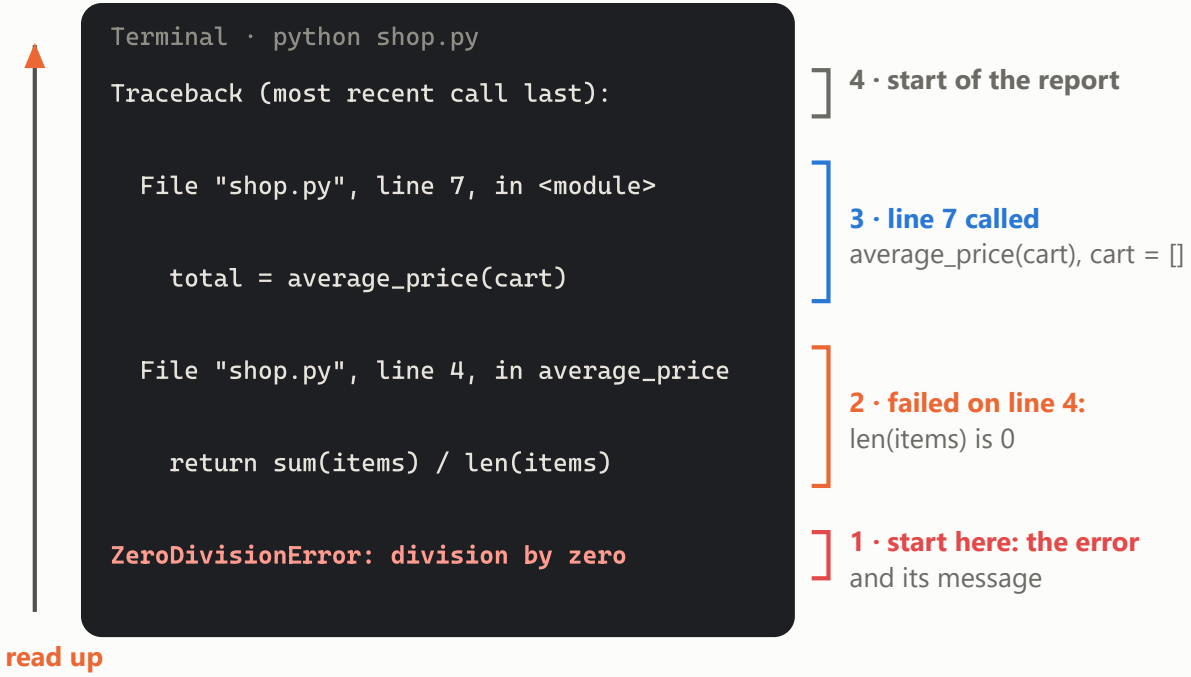

Already installed: import it and use it

KEY INSIGHT

**Before writing it yourself, check the standard library.**

# Read a traceback from the bottom up

- The last line names the error and what went wrong
- The lines above show the path, ending at the exact line that failed
- Common errors: `SyntaxError`, `NameError`, `TypeError`, `ValueError`, `IndexError`, `KeyError`, `ZeroDivisionError`

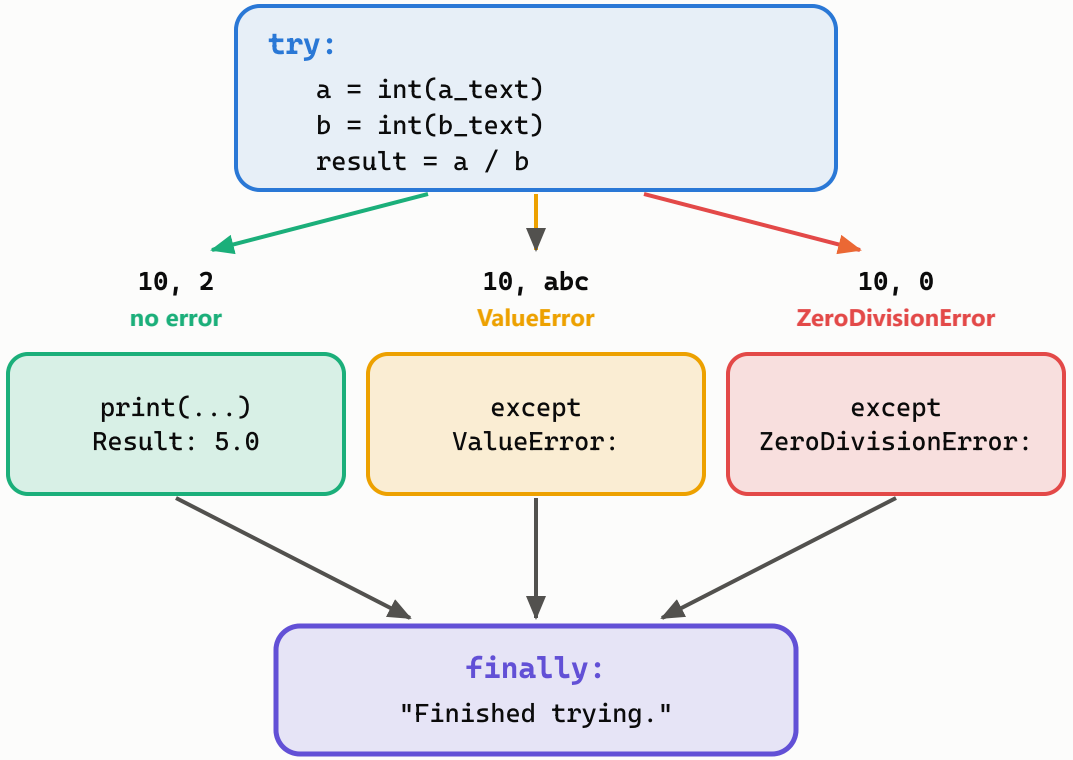



KEY INSIGHT

**Start at the last line, then go to the line number it points to.**

# try / except lets a program recover instead of crashing

- Put code that might fail inside `try`
- An error jumps to the `except` block that names its type, such as `except ValueError:`
- `finally` runs at the end every time, error or not
- `10, 2` → `Result: 5.0` · `10, abc` → ValueError · `10, 0` → ZeroDivisionError




every path ends in finally; the program carries on

KEY INSIGHT

**Plan for the errors you expect, and the program keeps going.**

# The final project: a quiz game that uses every lesson

| In the game | Lesson |
|---|---|
| `import random`, `import datetime as dt` | 8 · Modules |
| `QUESTIONS` : a list of dicts | 6 · Data Structures |
| `ask()`, `play()`, `main()` | 7 · Functions |
| `while True` … `break`, `try` / `except ValueError` | 5 · Control Flow, 9 · Errors |
| `answer.strip().lower()`, f-strings, `score / rounds * 100` | 4 · Strings, 3 · Operators |

```
$ python quiz_game.py

Welcome to the Python quiz!

How many questions (1–4)? abc

Please type a number.

How many questions (1–4)? 4

What is 7 * 6? 42

Correct!

…

You scored 3/4 (75%)
```

9 · try / except

5 · while True loop

6 · list of dicts

4 · strip().lower()

3 · score / rounds

KEY INSIGHT

**Small, simple pieces combine into a real program.**

# Key takeaways

1. A program runs line by line, top to bottom; `print()` shows values.
2. A variable is a name for a value; every value has a type.
3. `input()` gives text: convert before doing maths.
4. Expressions follow precedence: `2 + 3 * 4` is 14.
5. Strings are indexed from 0; slices stop before the stop index.
6. `if` / `elif` / `else` runs the first true branch; loops repeat a block.
7. list, tuple, dict and set each fit a different job.
8. Functions: define once, call often; `return` hands back a value.
9. `import` ready-made modules instead of writing them.
10. Read tracebacks from the bottom; `try` / `except` recovers.

# Try it yourself: free course, free certificate

**Introduction to Python**: a free, self-paced interactive course

`ibrahimsobh.github.io/introduction-to-python`

- Every idea in this talk has a live demo you can drive: step-through programs, a type explorer, an expression evaluator, a string slicer, a collections playground, a traceback reader and a quiz game
- No sign-up, runs in your browser. Pass the final quiz (8 of 10) for a free certificate

**Source on GitHub** · ⭐ Star the repo if you found it useful.




**KEY INSIGHT**

**You remember what you try. Pick one demo tonight.**